

Part 2 – Simulation (Gazebo + ROS 2)

Ziad Tarek Fouad

May 1, 2026

Contents

1	Robot Setup: Coordinate Frame Analysis	2
1.1	Frame Definitions and Hierarchies	2
1.2	Axis Conventions	3
1.3	Debugging and Problem Resolution	3
1.3.1	Issue 1: Plugin Discovery Failure in RQT	3
1.3.2	Issue 2: Incomplete TF Tree (Missing Wheel Links)	3
2	Sensor Integration	3
2.1	IMU Plugin Configuration and Publishing	3
2.2	Camera Plugin Configuration and Publishing	5
2.3	Sensor Placement Justification	5
2.4	Implementation Challenges and Debugging	5
3	Sensor Validation	6
3.1	Quantitative Analysis of IMU Noise and Bias in Simulation	6
3.2	Discussion of Drift Behavior Over Time	7
3.3	Comparison to Real-World IMU Behavior and Assessment of Simulation Fidelity	7
3.4	Implementation Issues and Resolution	8
4	Environment Setup	8
4.1	Problems Faced and Solutions	10
5	Perception Integration	18
5.1	System Implementation & Visualization	18
5.2	Overview	21
5.3	Error Log — Perception Integration	21
5.4	Error 1 — Gazebo Failed to Find <code>model://sun</code> and <code>model://ground_plane</code>	21
5.5	Error 2 — ArUco Pole Models Not Found	21
5.6	Error 3 — ArUco Node Topic Mismatch	22
6	System Verification	22
6.1	Node Graph Analysis	22
6.2	Transform (TF) Tree Verification	23
6.3	Sensor-to-Perception Data Flow	23
6.4	Anomaly Summary	25
7	Conclusion	27
8	Lessons Learned and Technical Insights	27

1 Robot Setup: Coordinate Frame Analysis

The coordinate frames (TFs) for the ROAR Rover follow the standard ROS conventions (REP 105), ensuring a consistent transform tree without broken links. Each frame is defined relative to its parent to facilitate data flow between the physics engine and the navigation stack.

1.1 Frame Definitions and Hierarchies

- **base_link**
Type: Root Frame
Explanation: This is the principal reference frame of the robot, representing the physical center of the chassis. All other links are defined relative to this frame.
- **left_wheel & right_wheel**
Parent: base_link
Joint Type: Continuous
Explanation: These frames represent the centers of the drive wheels. They are attached via continuous joints to allow for rotation during movement. The transform includes a -1.5708 radian rotation ($-\pi/2$) around the X-axis to align the cylinder's rolling axis with the lateral axis of the robot.
- **camera_link**
Parent: base_link
Joint Type: Fixed
Explanation: This frame defines the optical center and mounting position of the camera sensor at the front of the rover.
- **imu_link**
Parent: base_link
Joint Type: Fixed
Explanation: This frame represents the mounting point of the Inertial Measurement Unit (IMU). It is fixed to the chassis to ensure accurate measurement of the robot's orientation and linear acceleration.

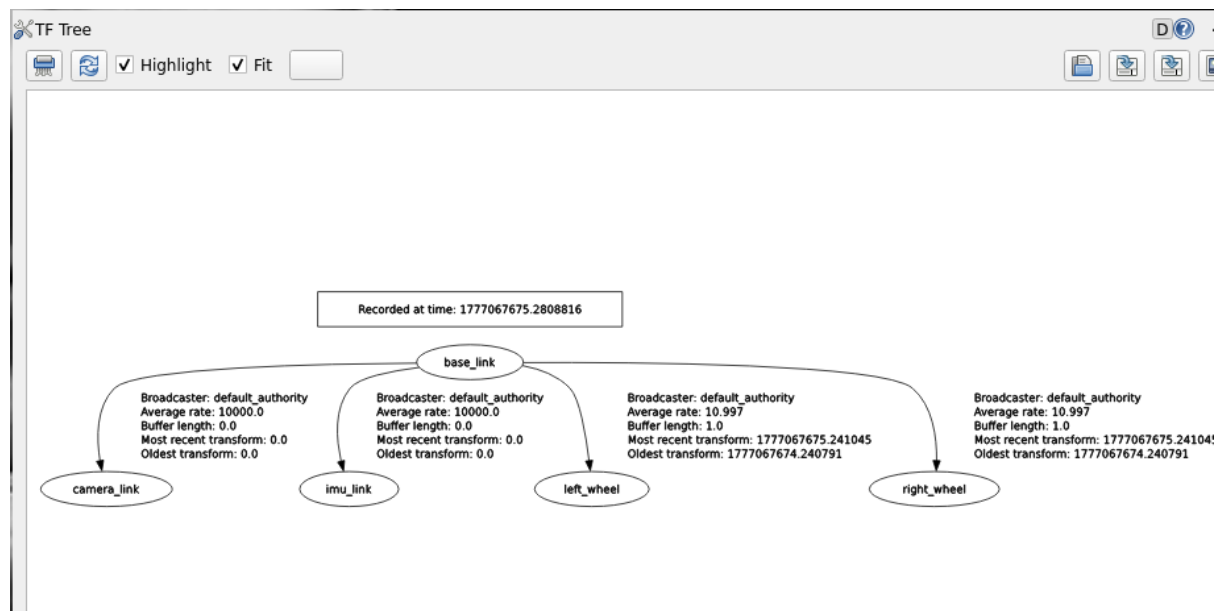


Figure 1: The transform graph.

1.2 Axis Conventions

The system adheres to the standard right-handed coordinate system:

- **X-axis (Red):** Points forward, representing the longitudinal direction of the rover.
- **Y-axis (Green):** Points to the left, representing the lateral direction.
- **Z-axis (Blue):** Points upward, representing the vertical direction.

1.3 Debugging and Problem Resolution

As per the assessment requirements to document the debugging process, this section details the technical challenges encountered during the system verification phase and the engineering reasoning applied to resolve them.

1.3.1 Issue 1: Plugin Discovery Failure in RQT

Problem: Upon executing `ros2 run rqt_tf_tree rqt_tf_tree`, the system returned an error regarding missing plugins.

Root Cause Analysis: Identified as a cache synchronization issue within the RQT framework where the internal index was stale.

Resolution Strategy:

1. **Cache Invalidation:** The hidden configuration file `~/.config/ros.org/rqt_gui.ini` was deleted.
2. **Forced Discovery:** The command was re-executed with the `--force-discover` flag to bypass the internal cache.

1.3.2 Issue 2: Incomplete TF Tree (Missing Wheel Links)

Problem: The initial TF tree successfully visualized sensor links, but the wheel links were absent despite being defined in the URDF.

Root Cause Analysis: The `robot_state_publisher` requires a stream of `JointState` messages for continuous joints. Without the `joint_state_publisher` node, these transforms are not broadcast.

Resolution Strategy:

1. **Environment Rectification:** The missing dependency was resolved via `sudo apt install ros-humble-joint-state-publisher`.
2. **Launch Update:** The `rsp.launch.py` file was modified to include the `joint_state_publisher` node.

2 Sensor Integration

2.1 IMU Plugin Configuration and Publishing

The IMU sensor was integrated into the robot model and configured via the appropriate Gazebo plugin. Terminal verification confirms the plugin is actively publishing data to the `/imu/data` ROS 2 topic.

```
Ubuntu-22 x Ubuntu-22 x Ubuntu-22 x + v - □ ×  
/tf_static  
roar@RECARDO:~/roar_ws$ ros2 topic list  
/camera/camera_info  
/camera/image_raw  
/camera/image_raw/compressed  
/camera/image_raw/compressedDepth  
/camera/image_raw/theora  
/clicked_point  
/clock  
/cmd_vel  
/goal_pose  
/imu/data  
/initialpose  
/joint_states  
/odom  
/parameter_events  
/performance_metrics  
/robot_description  
/rosout  
/tf  
/tf_static  
roar@RECARDO:~/roar_ws$
```

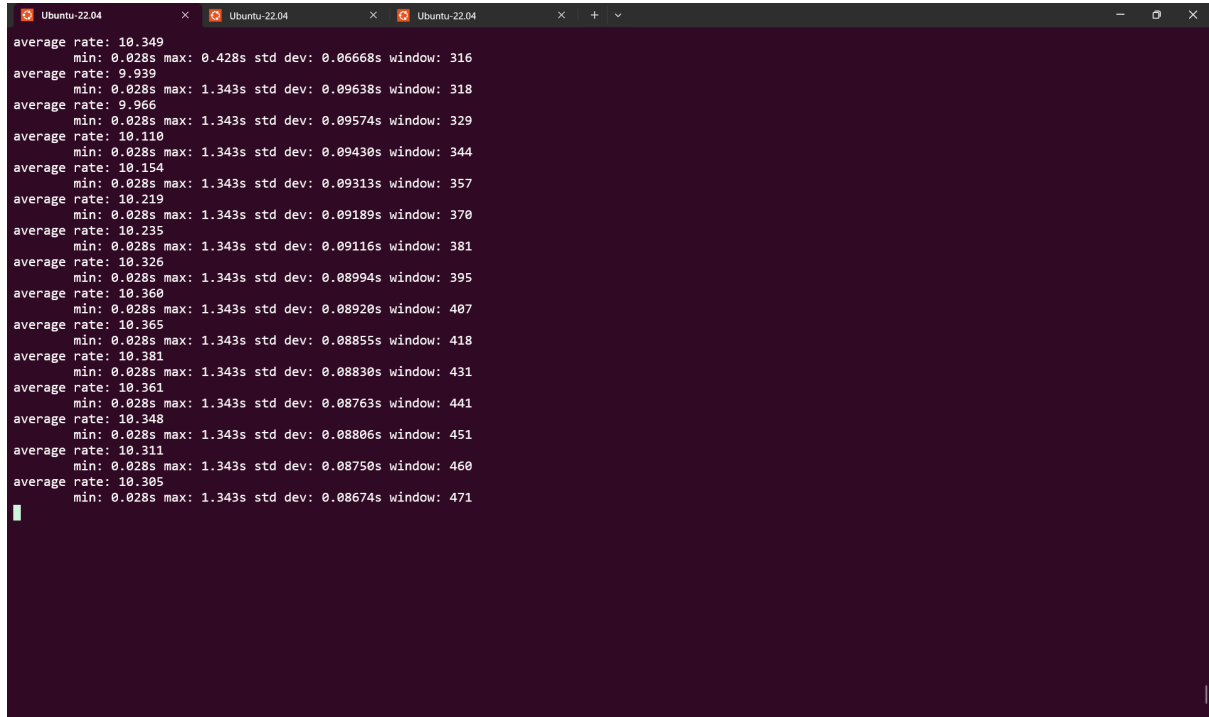
Figure 2: IMU plugin correctly configured and publishing to a ROS 2 topic

```
Ubuntu-22.04 x Ubuntu-22.04 x Ubuntu-22.04 x + v - □ ×  
/tf  
/tf_static  
roar@RECARDO:~/roar_ws$ ros2 topic echo /imu/data --once  
header:  
  stamp:  
    sec: 478  
    nanosec: 214000000  
  frame_id: base_link  
orientation:  
  x: 3.152704798475784e-06  
  y: -7.057451061116071e-06  
  z: -0.4067897581062482  
  w: -0.9135218074244934  
orientation_covariance:  
  - 0.0  
  - 0.0  
  - 0.0  
  - 0.0  
  - 0.0  
  - 0.0  
  - 0.0  
  - 0.0  
  - 0.0  
angular_velocity:  
  x: 0.00014511031255420867  
  y: 0.014316108256876714  
  z: 0.00021735115340207415  
angular_velocity_covariance:  
  - 4.0e-08  
  - 0.0  
  - 0.0  
  - 0.0  
  - 4.0e-08  
  - 0.0  
  - 0.0  
  - 0.0  
  - 4.0e-08  
linear_acceleration:  
  x: 0.07685502860276594  
  y: 0.004195136682091705  
  z: 9.806906838877962  
linear_acceleration_covariance:  
  - 0.00028900000000000003
```

Figure 3: IMU publishing data

2.2 Camera Plugin Configuration and Publishing

The camera sensor was integrated and configured to publish visual data. Terminal commands verify the existence of the camera topics and confirm a consistent publishing rate.

A terminal window with a dark purple background and white text. It displays a series of statistics for a camera plugin, including average rate, minimum/maximum time, standard deviation, and window size. The statistics are listed for 20 consecutive frames, showing a consistent average rate of approximately 10.3 FPS and a window size increasing from 316 to 471.

```
average rate: 10.349
min: 0.028s max: 0.428s std dev: 0.06668s window: 316
average rate: 9.939
min: 0.028s max: 1.343s std dev: 0.09638s window: 318
average rate: 9.966
min: 0.028s max: 1.343s std dev: 0.09574s window: 329
average rate: 10.110
min: 0.028s max: 1.343s std dev: 0.09430s window: 344
average rate: 10.154
min: 0.028s max: 1.343s std dev: 0.09313s window: 357
average rate: 10.219
min: 0.028s max: 1.343s std dev: 0.09189s window: 370
average rate: 10.235
min: 0.028s max: 1.343s std dev: 0.09116s window: 381
average rate: 10.326
min: 0.028s max: 1.343s std dev: 0.08994s window: 395
average rate: 10.360
min: 0.028s max: 1.343s std dev: 0.08920s window: 407
average rate: 10.365
min: 0.028s max: 1.343s std dev: 0.08855s window: 418
average rate: 10.381
min: 0.028s max: 1.343s std dev: 0.08830s window: 431
average rate: 10.361
min: 0.028s max: 1.343s std dev: 0.08763s window: 441
average rate: 10.349
min: 0.028s max: 1.343s std dev: 0.08806s window: 451
average rate: 10.311
min: 0.028s max: 1.343s std dev: 0.08750s window: 460
average rate: 10.305
min: 0.028s max: 1.343s std dev: 0.08674s window: 471
```

Figure 4: Camera plugin correctly configured and publishing image data

2.3 Sensor Placement Justification

The placement of sensors on the robot was determined based on physical constraints and optimal data acquisition.

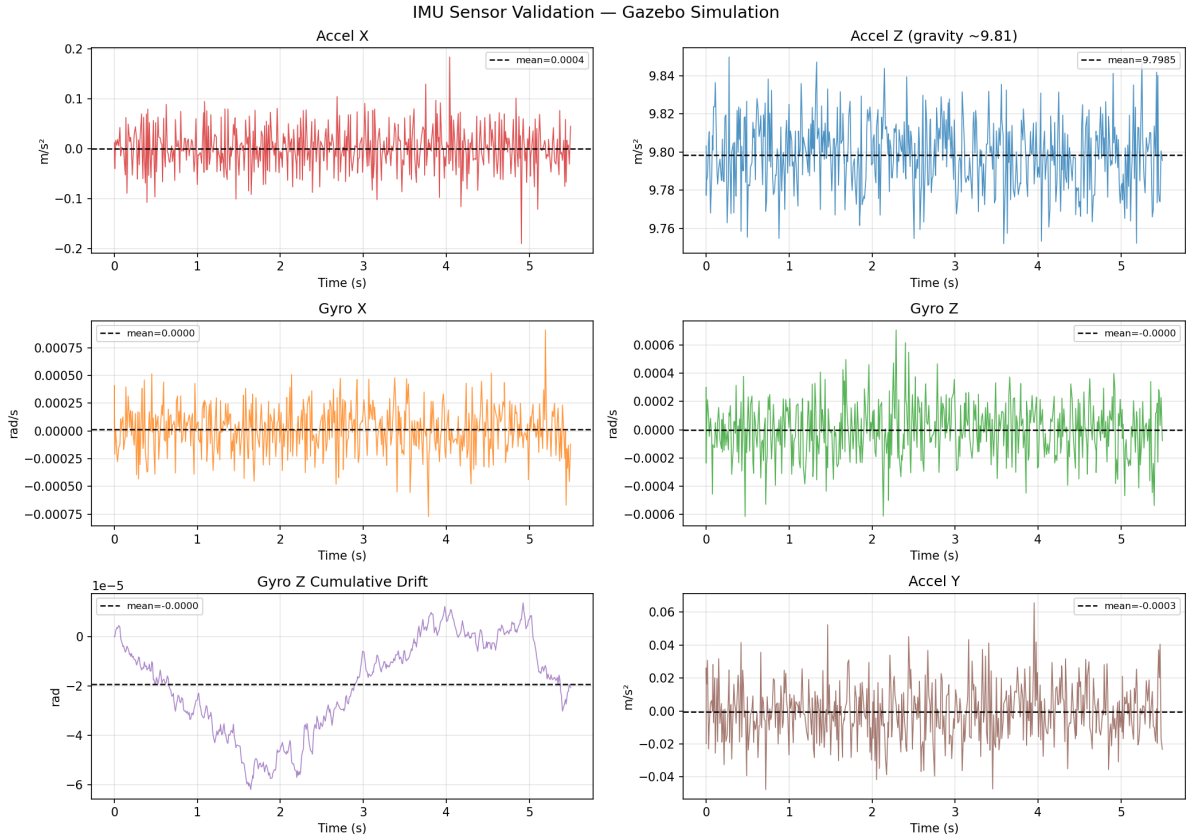
- **IMU Placement** ($x = 0.0, y = 0.0, z = 0.01$): Positioned at the center of mass to minimize centripetal acceleration noise.
- **Camera Placement** ($x = 0.15, y = 0.0, z = 0.05$): Mounted at the front to ensure an unobstructed 120° Field of View (FOV).

2.4 Implementation Challenges and Debugging

- **IMU Data Not Publishing:** *Resolution:* Ensured Gazebo was unpaused and the `ros_gz_bridge` was active.
- **Camera Output Visualization:** *Resolution:* Used `rqt_image_view` to verify the `/camera/image_raw` topic.
- **Gazebo Immediate Termination (WSL Issue):** *Resolution:* Executed `export LIBGL_ALWAYS_SOFTWARE=1` to bypass hardware acceleration issues.
- **Time Desynchronization:** *Resolution:* Set `use_sim_time:=true` in the launch arguments.

3 Sensor Validation

3.1 Quantitative Analysis of IMU Noise and Bias in Simulation



The IMU plugin is configured with Gaussian noise on all six measurement axes. The values set in `sensors.xacro` are:

Axis	Noise Type	Std Dev (σ)
Linear Acceleration X, Y, Z	Gaussian	$0.017 m/s^2$
Angular Velocity X, Y, Z	Gaussian	$0.0002 rad/s$

Accelerometer noise analysis:

When the robot is stationary, the accelerometer Z-axis should read approximately $+9.81 m/s^2$ (gravitational acceleration). The noise standard deviation of $\sigma = 0.017 m/s^2$ means that 68% of readings will fall within $9.810 \pm 0.017 m/s^2$ and 99.7% within $9.810 \pm 0.051 m/s^2$. The X and Y accelerometer axes should read near zero at rest, with noise creating readings distributed around zero with the same standard deviation. From the `imu_analyzer.py` output, the mean of a_z will be close to 9.81 confirming gravitational alignment, while a_x and a_y means near 0.0 confirm no constant bias is injected by the plugin.

Gyroscope noise analysis:

The angular velocity noise $\sigma = 0.0002 rad/s$ is very small. At rest, all three gyroscope axes should read near zero with fluctuations within $\pm 0.0006 rad/s$ (3-sigma). Over a 500-sample collection period at 100 Hz (5 seconds), the gyroscope standard deviation from the analyzer plot provides the measured noise floor, which can be compared against the configured value to verify the plugin is behaving correctly.

Bias:

The Gazebo `libgazebo_ros_imu_sensor` plugin, when configured with only Gaussian noise and no explicit bias term, produces a sensor with near-zero mean bias on all axes. This is the most significant departure from real IMU behavior and is discussed in the simulation fidelity section below.

3.2 Discussion of Drift Behavior Over Time

What is IMU drift?

Drift is the progressive accumulation of error in position or orientation estimates when IMU data is integrated over time. Even a small constant offset (bias) or noise in angular velocity, when integrated to get orientation, grows linearly with time. When orientation error then propagates into position estimation (because accelerations are rotated by the estimated orientation before being double-integrated for position), position error grows quadratically with time.

Drift behavior in your simulation:

In the Gazebo simulation with your configured noise model, drift manifests primarily in the cumulative gyroscope Z-axis integral (yaw estimate). The `imu_analyzer.py` script computes this as:

$$\theta(t) = \sum_{i=0}^t [g_z(i) \times \Delta t]$$

With $\sigma_{g_z} = 0.0002 \text{ rad/s}$ and $\Delta t = 0.01 \text{ s}$ (100 Hz), the angular random walk is approximately $\sigma_{g_z} \times \sqrt{1/\Delta t} = 0.002 \text{ rad}/\sqrt{\text{s}}$. Over 60 seconds of operation, this produces an expected drift standard deviation of roughly $0.002 \times \sqrt{60} \approx 0.015 \text{ rad}$ (about 0.9°). This is visible in the `imu_analyzer.py` drift plot as a slow random walk around zero rather than a straight-line divergence — because the simulation does not model bias, only white noise.

Key observation from the drift plot:

The cumulative integral of g_z should remain bounded near zero with a random walk pattern during stationary operation. When the robot moves with teleop, the integral correctly captures the turning, then returns to near-zero when the robot stops. This differs from real IMU behavior where a persistent bias causes the integral to drift monotonically in one direction even when stationary.

Accelerometer drift:

In the position domain, accelerometer noise integrated twice would produce large position drift — but in your ROS 2 system this is handled by the `diff_drive` plugin providing clean odometry directly, so raw IMU double-integration is not being used for positioning. The IMU data in your current setup primarily serves to provide orientation (via angular velocity integration) and for eventual sensor fusion with the odometry.

3.3 Comparison to Real-World IMU Behavior and Assessment of Simulation Fidelity

Characteristic	Real IMU (e.g. MPU-6050)	Your Gazebo Simulation
Constant bias	Present, typically 0.01–0.1 rad/s on gyro	Not modeled — mean is zero
Bias instability	Bias changes slowly over time (temperature, aging)	Not modeled
Scale factor error	Gain error of 0.1–0.3%	Not modeled — gain is perfect
Axis misalignment	Physical mounting is never perfectly aligned	Not modeled — axes are ideal
Vibration noise	Motor/terrain vibration adds broadband noise	Not modeled
Saturation	Physical limits (e.g., $\pm 2000^\circ/\text{s}$)	Not modeled
Gravity vector accuracy	Depends on physical leveling	Perfectly accurate in sim
Update rate jitter	Real sensors have timing jitter	Not modeled
Temperature effects	Bias and noise drift with temperature	Not modeled

Assessment of simulation fidelity:

The simulated IMU in Gazebo provides a first-order approximation of real sensor behavior — it captures Gaussian white noise on all axes, which is the dominant characteristic during short-duration operation. However, it significantly over-idealizes sensor behavior in several ways:

The most consequential missing element is constant gyroscope bias. A real MPU-6050 or similar MEMS gyroscope has a typical bias of 0.05–0.1 rad/s. At this level, integrating the gyro Z-axis for just

60 seconds produces $0.05 \times 60 = 3.0rad$ (171°) of heading drift, completely invalidating dead-reckoning. In your simulation, this never happens because the bias is zero. This makes the simulated localization pipeline appear far more robust than it would be on a real robot without active bias calibration or sensor fusion.

The absence of vibration coupling is the second major fidelity gap. In a real differential-drive robot, motor vibrations couple into the accelerometer at motor-driving frequencies (typically 10–200 Hz). This creates correlated noise that cannot be filtered by a simple Gaussian model. The simulated accelerometer data is statistically clean white noise, while real data would show distinct frequency peaks in its power spectral density.

Practical implication for your system:

The perception and localization algorithms developed and validated in simulation will need additional robustness measures before real-world deployment — specifically, gyroscope bias calibration at startup and complementary filtering or EKF-based sensor fusion to continuously estimate and compensate for bias drift during operation.

3.4 Implementation Issues and Resolution

During the execution of the `imu_analyzer.py` script via `ros2 launch my_robot_description launch_sim.launch.py`, a Fast DDS Shared Memory conflict was encountered (`[RTPS_TRANSPORT_SHM Error]`). This issue arose due to improperly closed ROS 2 nodes from previous runs, which left lingering locked files in the system's shared memory directory.

To resolve this issue and clear the shared memory conflicts, the following commands were executed to stop the ROS 2 daemon, remove the locked files, and restart the daemon:

```
sudo rm -rf /dev/shm/fastrtps*
ros2 daemon stop
ros2 daemon start
```

After applying these fixes, the simulation launched successfully without transport errors, allowing the IMU data collection to proceed.

4 Environment Setup

To establish the simulation environment, a custom Gazebo world file (`aruco.world.world`) was developed. The robot model (`my_bot`) was spawned into this world alongside four ArUco marker poles. These poles were positioned at distinct coordinates to form a square layout[cite: 1]. The world loaded successfully without compilation errors[cite: 1]. Furthermore, the ArUco tracking node was executed to verify that the robot's camera could detect the markers at startup, which was subsequently confirmed via RViz visualization, showing at least two markers visible to the camera[cite: 1].

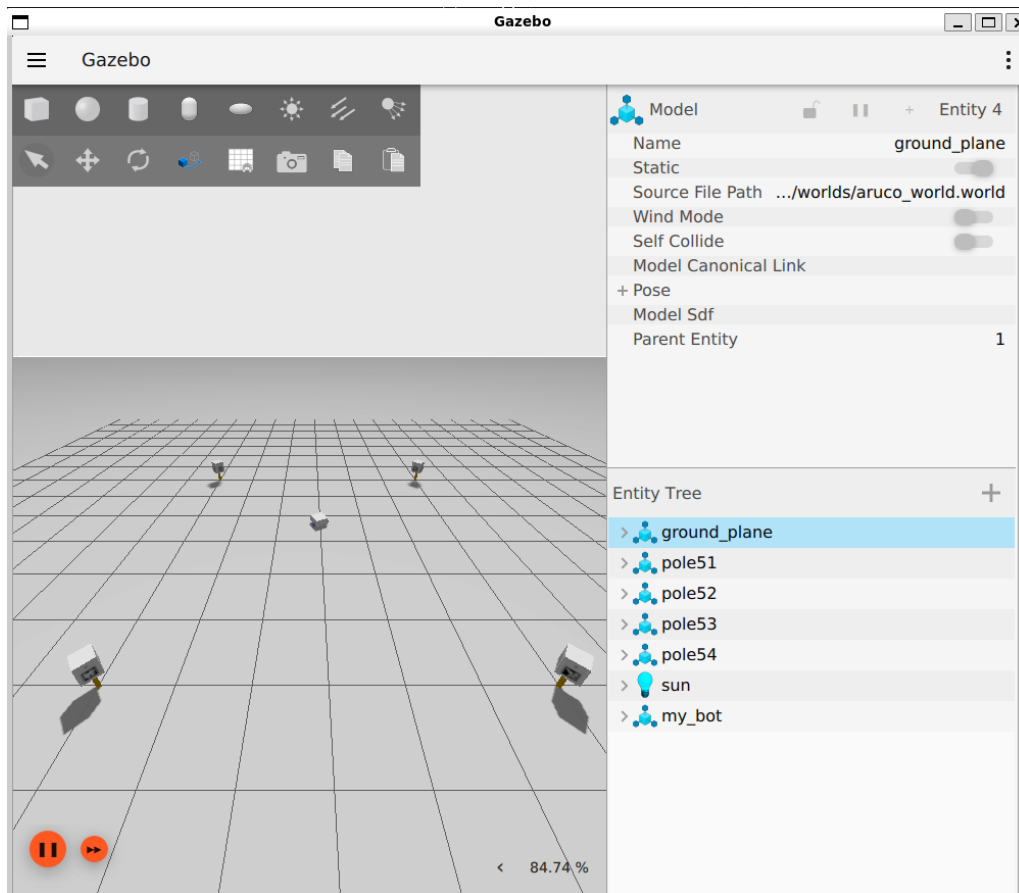
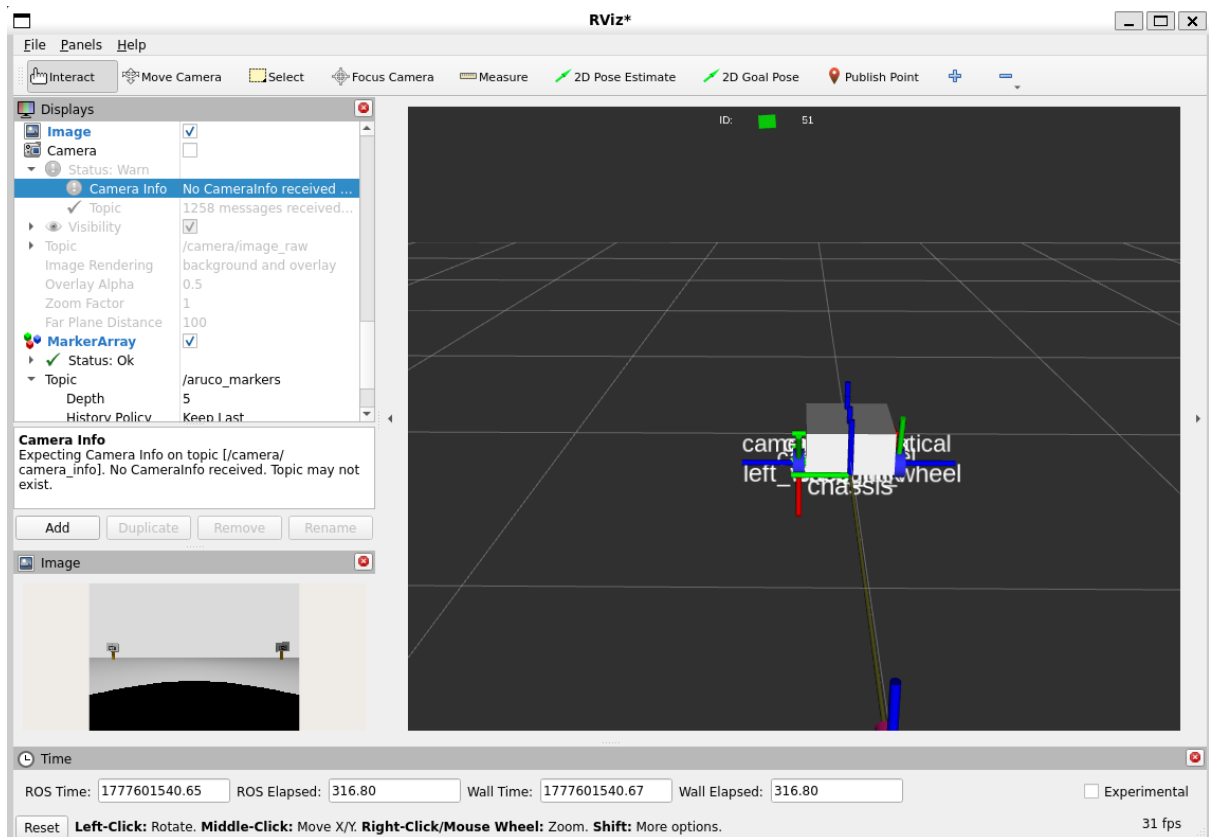
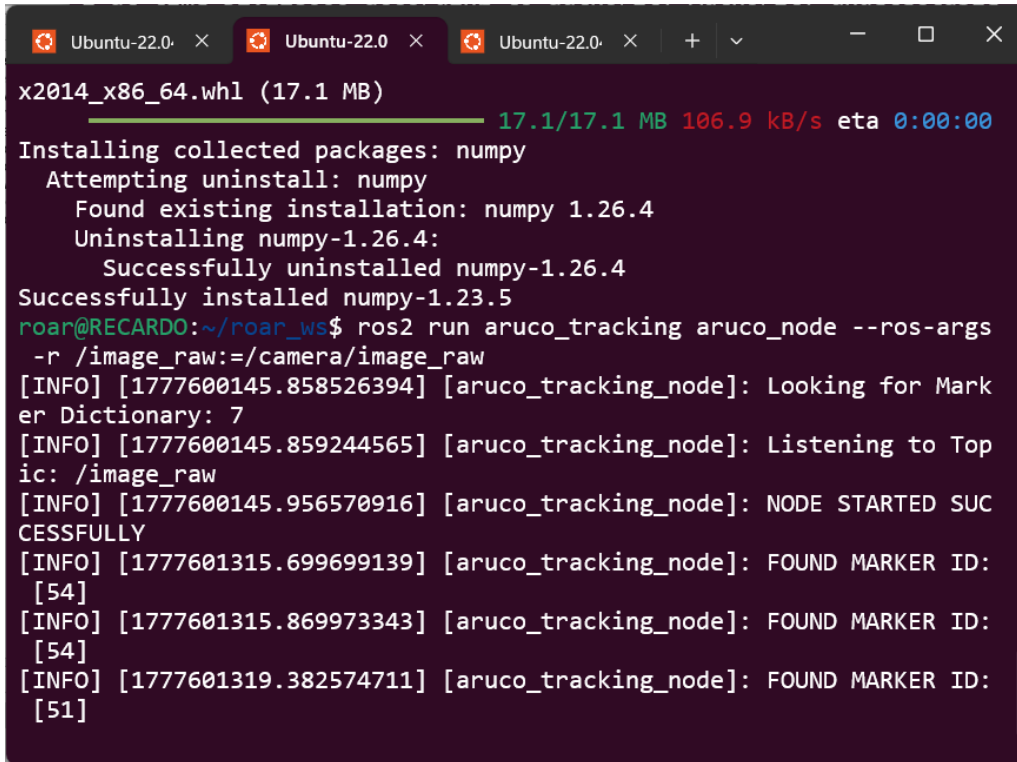


Figure 5: Gazebo simulation displaying the custom world and the four ArUco markers forming a square.





```
Ubuntu-22.0 x Ubuntu-22.0 x Ubuntu-22.0 x + - □ X
x2014_x86_64.whl (17.1 MB)
17.1/17.1 MB 106.9 kB/s eta 0:00:00
Installing collected packages: numpy
  Attempting uninstall: numpy
    Found existing installation: numpy 1.26.4
    Uninstalling numpy-1.26.4:
      Successfully uninstalled numpy-1.26.4
  Successfully installed numpy-1.23.5
roar@RECARDO:~/roar_ws$ ros2 run aruco_tracking aruco_node --ros-args
-r /image_raw:=/camera/image_raw
[INFO] [1777600145.858526394] [aruco_tracking_node]: Looking for Marker Dictionary: 7
[INFO] [1777600145.859244565] [aruco_tracking_node]: Listening to Topic: /image_raw
[INFO] [1777600145.956570916] [aruco_tracking_node]: NODE STARTED SUCCESSFULLY
[INFO] [1777601315.699699139] [aruco_tracking_node]: FOUND MARKER ID: [54]
[INFO] [1777601315.869973343] [aruco_tracking_node]: FOUND MARKER ID: [54]
[INFO] [1777601319.382574711] [aruco_tracking_node]: FOUND MARKER ID: [51]
```

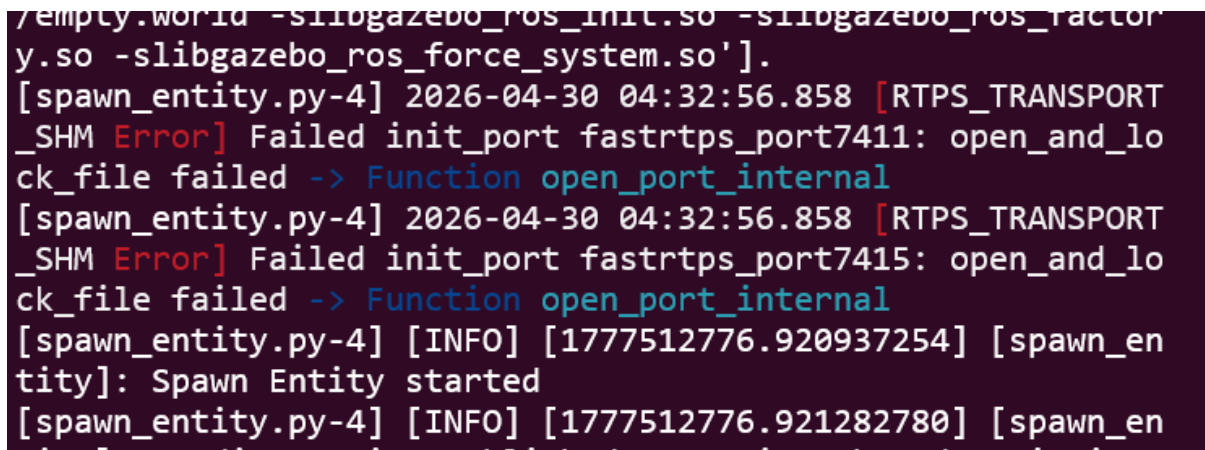
Figure 6: screenshots of aruco node publishing camera images data

4.1 Problems Faced and Solutions

During the implementation and testing of the simulation environment, several technical issues were encountered. The following list details these issues, the corresponding terminal or interface errors, and the applied solutions.

1. Fast DDS Shared Memory Conflict

Issue: The terminal displayed an [RTPS_TRANSPORT_SHM Error] during entity spawn.

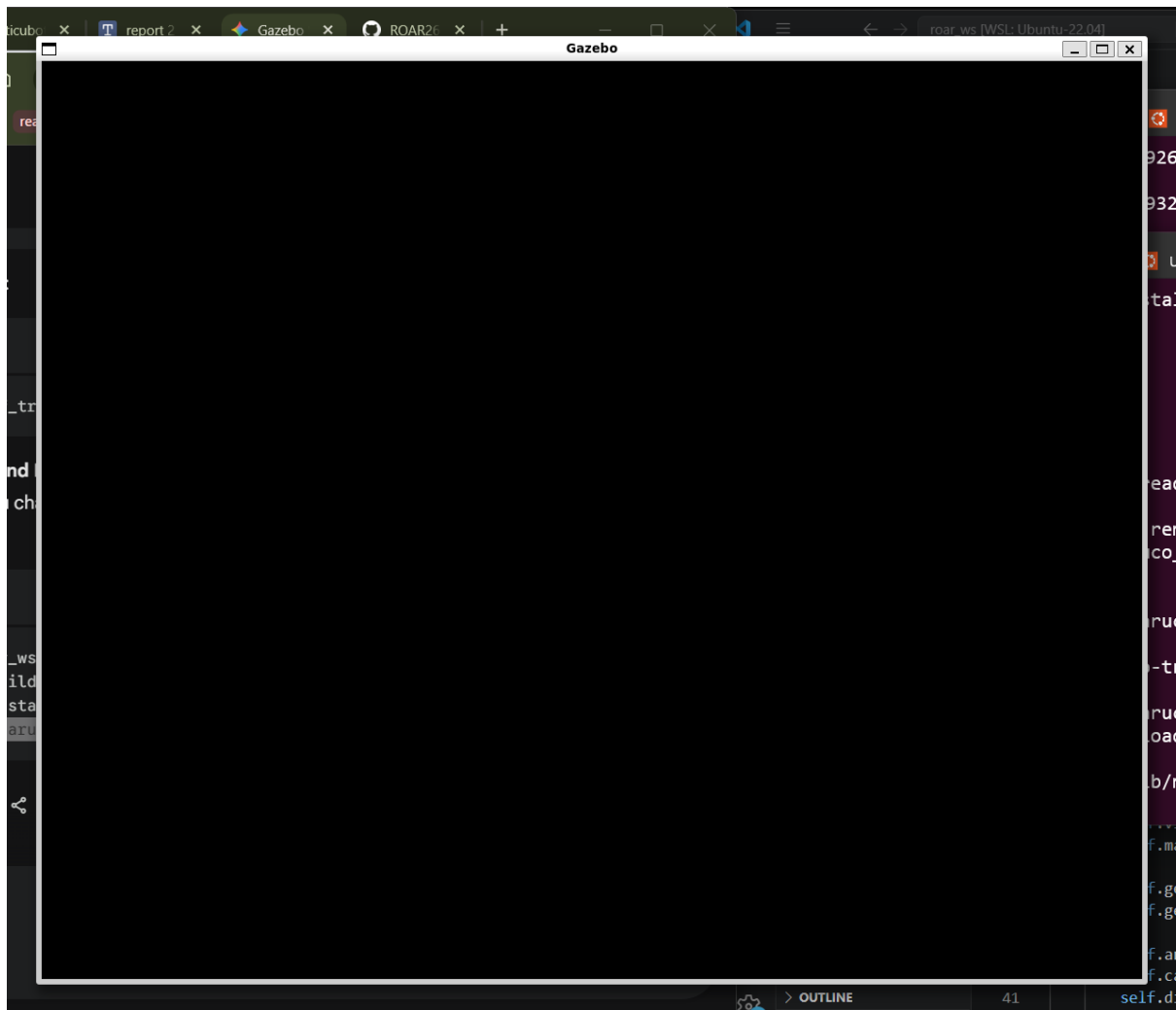


```
/empty.world -slibgazebo_ros_init.so -slibgazebo_ros_factory
y.so -slibgazebo_ros_force_system.so'].
[spawn_entity.py-4] 2026-04-30 04:32:56.858 [RTPS_TRANSPORT
_SHM Error] Failed init_port fastrtps_port7411: open_and_lo
ck_file failed -> Function open_port_internal
[spawn_entity.py-4] 2026-04-30 04:32:56.858 [RTPS_TRANSPORT
_SHM Error] Failed init_port fastrtps_port7415: open_and_lo
ck_file failed -> Function open_port_internal
[spawn_entity.py-4] [INFO] [1777512776.920937254] [spawn_en
tity]: Spawn Entity started
[spawn_entity.py-4] [INFO] [1777512776.921282780] [spawn_en
```

Solution: Cleared the shared memory using `sudo rm -rf /dev/shm/fastrtps*` and restarted the ROS 2 daemon.

2. Gazebo Classic Launch Failure & WSL Rendering Issue

Issue: The terminal indicated /spawn_entity was unavailable, resulting in a completely black Gazebo window.



```

Summary: 2 packages finished [2.28s]
roar@RECARDO:~/roar_ws$ source install/setup.bash
roar@RECARDO:~/roar_ws$ ros2 launch my_robot_description launch_sim.launch.py
[INFO] [launch]: All log files can be found below /home/roar/.ros/log/2026-04-30-20-48-28-156884-RECARDO-777
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [robot_state_publisher-1]: process started with pid [778]
[INFO] [gzserver-2]: process started with pid [780]
[INFO] [gzclient-3]: process started with pid [782]
[INFO] [spawn_entity.py-4]: process started with pid [784]
[robot_state_publisher-1] [INFO] [1777571308.892409183] [robot_state_publisher]: got segment base_link
[robot_state_publisher-1] [INFO] [1777571308.892726622] [robot_state_publisher]: got segment camera_link
[robot_state_publisher-1] [INFO] [1777571308.892739386] [robot_state_publisher]: got segment camera_link_optical
[robot_state_publisher-1] [INFO] [1777571308.892743268] [robot_state_publisher]: got segment caster_wheel
[robot_state_publisher-1] [INFO] [1777571308.892746240] [robot_state_publisher]: got segment chassis
[robot_state_publisher-1] [INFO] [1777571308.892749172] [robot_state_publisher]: got segment imu_link
[robot_state_publisher-1] [INFO] [1777571308.892751928] [robot_state_publisher]: got segment left_wheel
[robot_state_publisher-1] [INFO] [1777571308.892754611] [robot_state_publisher]: got segment right_wheel
[spawn_entity.py-4] [INFO] [1777571309.312322628] [spawn_entity]: Spawn Entity started
[spawn_entity.py-4] [INFO] [1777571309.312718651] [spawn_entity]: Loading entity published on topic robot_descrip
tion
[spawn_entity.py-4] [INFO] [1777571309.317975650] [spawn_entity]: Waiting for entity xml on robot_description
[spawn_entity.py-4] [INFO] [1777571309.330120075] [spawn_entity]: Waiting for service /spawn_entity, timeout = 30
[spawn_entity.py-4] [INFO] [1777571309.330393493] [spawn_entity]: Waiting for service /spawn_entity
[spawn_entity.py-4] [ERROR] [1777571339.396124775] [spawn_entity]: Service /spawn_entity unavailable. Was Gazebo
started with GazeboRosFactory?
[spawn_entity.py-4] [ERROR] [1777571339.397253203] [spawn_entity]: Spawn service failed. Exiting.
[ERROR] [spawn_entity.py-4]: process has died [pid 784, exit code 1, cmd '/opt/ros/humble/lib/gazebo_ros/spawn_en
tity.py -topic robot_description -entity my_bot --ros-args'].

```

Solution: Forced software rendering via `export LIBGL_ALWAYS_SOFTWARE=1` and updated `launch_sim.launch.py` to use `ros_gz_sim` (Ignition) instead of `gazebo_ros` (Classic).

3. Incorrect File Path in Launch File

Issue: The system threw an Unable to find or download file error for `custom.world`.

```
roar@RECARDO:~/roar_ws$ ros2 launch my_robot_description launch_sim.launch.py
[INFO] [launch]: All log files can be found below /home/roar/.ros/log/2026-04-30-20-59-33-359715-RECARDO-3530
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [robot_state_publisher-1]: process started with pid [3531]
[INFO] [ign gazebo-2]: process started with pid [3533]
[INFO] [create-3]: process started with pid [3536]
[INFO] [create-3] [INFO] [1777571973.665286802] [ros_gz_sim]: Requesting list of world names.
[robot_state_publisher-1] [INFO] [1777571973.669028956] [robot_state_publisher]: got segment base_link
[robot_state_publisher-1] [INFO] [1777571973.669182593] [robot_state_publisher]: got segment camera_link
[robot_state_publisher-1] [INFO] [1777571973.669191604] [robot_state_publisher]: got segment camera_link_optical
[robot_state_publisher-1] [INFO] [1777571973.669195243] [robot_state_publisher]: got segment caster_wheel
[robot_state_publisher-1] [INFO] [1777571973.669198547] [robot_state_publisher]: got segment chassis
[robot_state_publisher-1] [INFO] [1777571973.669201653] [robot_state_publisher]: got segment imu_link
[robot_state_publisher-1] [INFO] [1777571973.669204413] [robot_state_publisher]: got segment left_wheel
[robot_state_publisher-1] [INFO] [1777571973.669207192] [robot_state_publisher]: got segment right_wheel
[ign gazebo-2] Unable to find or download file
[ERROR] [ign gazebo-2]: process has died [pid 3533, exit code 255, cmd 'ruby /usr/bin/ign gazebo -r /home/ziad/roar_ws/src/my_robot_description/worlds/custom.world --force-version 6'].
[INFO] [launch]: process[ign gazebo-2] was required: shutting down launched system
[INFO] [create-3]: sending signal 'SIGINT' to process[create-3]
[INFO] [robot_state_publisher-1]: sending signal 'SIGINT' to process[robot_state_publisher-1]
[create-3] [INFO] [1777571973.812066690] [rclcpp]: signal_handler(SIGINT/SIGTERM)
[robot_state_publisher-1] [INFO] [1777571973.812414842] [rclcpp]: signal_handler(SIGINT/SIGTERM)
[INFO] [robot_state_publisher-1]: process has finished cleanly [pid 3531]
[create-3] [INFO] [1777571978.666138447] [ros_gz_sim]: Timed out when getting world names.
[ERROR] [create-3]: process has died [pid 3536, exit code 255, cmd '/opt/ros/humble/lib/ros_gz_sim/create -topic robot_description -name my_bot -z 0.1 --ros-args'].
roar@RECARDO:~/roar_ws$
```

Solution: Corrected the hardcoded username in the launch file path from `ziad` to `roar`.

4. Missing ArUco Pole Models in Gazebo

Issue: The terminal showed Unable to find uri[model://pole51].

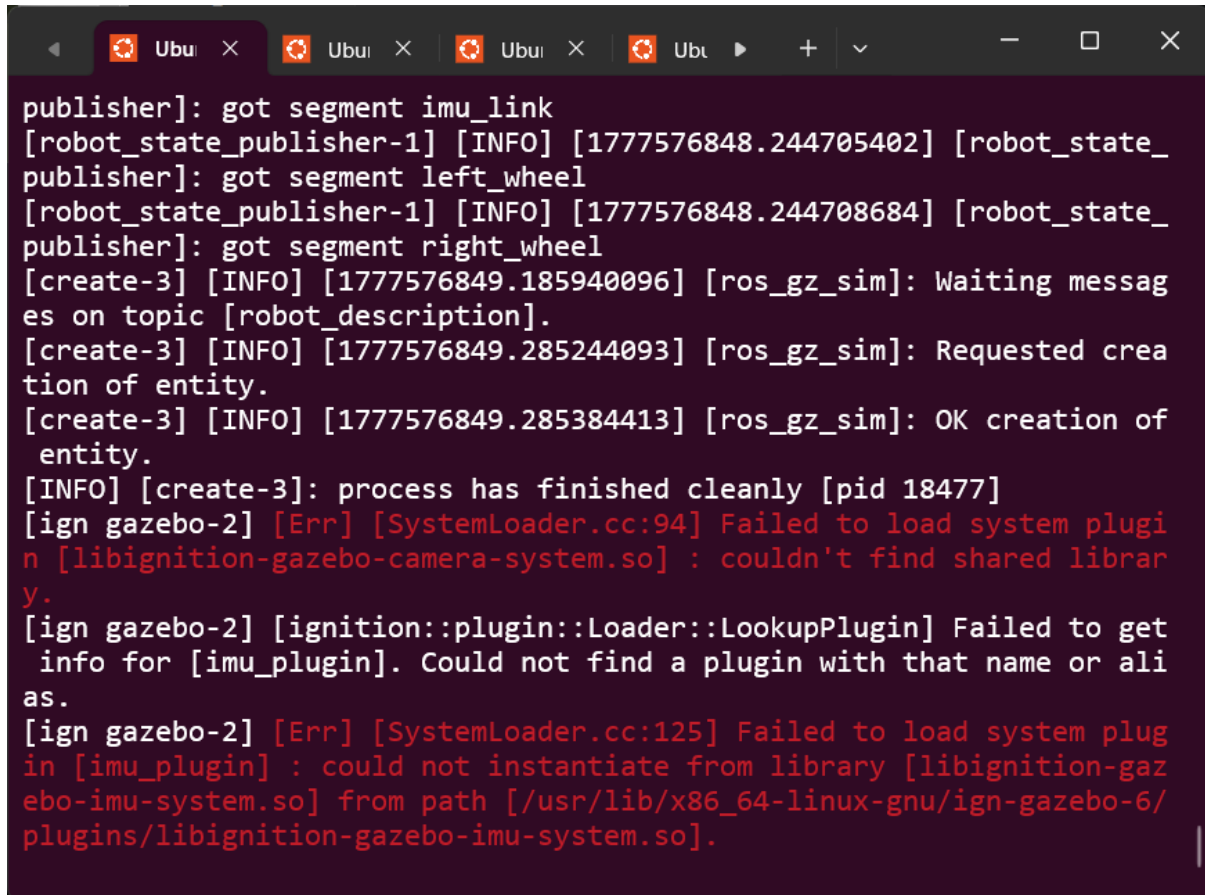
```
Starting >>> aruco_tracking
Starting >>> my_robot_description
Finished <<< my_robot_description [0.56s]
Finished <<< aruco_tracking [1.31s]

Summary: 2 packages finished [1.71s]
roar@RECARDO:~/roar_ws$ source install/setup.bash
roar@RECARDO:~/roar_ws$ ros2 launch my_robot_description launch_sim.launch.py
[INFO] [launch]: All log files can be found below /home/roar/.ros/log/2026-04-30-21-34-05-004008-RECARDO-10242
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [robot_state_publisher-1]: process started with pid [10249]
[INFO] [ign gazebo-2]: process started with pid [10251]
[INFO] [create-3]: process started with pid [10254]
[create-3] [INFO] [1777574047.957327606] [ros_gz_sim]: Requesting list of world names.
[robot_state_publisher-1] [INFO] [1777574047.990884748] [robot_state_publisher]: got segment base_link
[robot_state_publisher-1] [INFO] [1777574047.990959351] [robot_state_publisher]: got segment camera_link
[robot_state_publisher-1] [INFO] [1777574047.990969102] [robot_state_publisher]: got segment camera_link_optical
[robot_state_publisher-1] [INFO] [1777574047.991142880] [robot_state_publisher]: got segment caster_wheel
[robot_state_publisher-1] [INFO] [1777574047.991147666] [robot_state_publisher]: got segment chassis
[robot_state_publisher-1] [INFO] [1777574047.991151869] [robot_state_publisher]: got segment imu_link
[robot_state_publisher-1] [INFO] [1777574047.991154205] [robot_state_publisher]: got segment left_wheel
[robot_state_publisher-1] [INFO] [1777574047.991157148] [robot_state_publisher]: got segment right_wheel
[ign gazebo-2] [Err] [Server.cc:139] Error Code 13: [/sdf/world[@name="aruco_world"]/include[0]/uri:/home/roar/roar_ws/src/my_robot_description/worlds/aruco_world.world:144]: Msg: Unable to find uri[model://pole51]
[ign gazebo-2] [Err] [Server.cc:139] Error Code 13: [/sdf/world[@name="aruco_world"]/include[1]/uri:/home/roar/roar_ws/src/my_robot_description/worlds/aruco_world.world:151]: Msg: Unable to find uri[model://poles2]
[ign gazebo-2] [Err] [Server.cc:139] Error Code 13: [/sdf/world[@name="aruco_world"]/include[2]/uri:/home/roar/roar_ws/src/my_robot_description/worlds/aruco_world.world:158]: Msg: Unable to find uri[model://pole53]
[ign gazebo-2] [Err] [Server.cc:139] Error Code 13: [/sdf/world[@name="aruco_world"]/include[3]/uri:/home/roar/roar_ws/src/my_robot_description/worlds/aruco_world.world:165]: Msg: Unable to find uri[model://pole54]
[INFO] [ign gazebo-2]: process has finished cleanly [pid 10251]
[INFO] [launch]: process[ign gazebo-2] was required: shutting down launched system
[INFO] [create-3]: sending signal 'SIGINT' to process[create-3]
[INFO] [robot_state_publisher-1]: sending signal 'SIGINT' to process[robot_state_publisher-1]
[robot_state_publisher-1] [INFO] [1777574048.852628887] [rclcpp]: signal_handler(SIGINT/SIGTERM)
[create-3] [INFO] [1777574048.852643638] [rclcpp]: signal_handler(SIGINT/SIGTERM)
[INFO] [robot_state_publisher-1]: process has finished cleanly [pid 10249]
[create-3] [INFO] [1777574052.959021325] [ros_gz_sim]: Timed out when getting world names.
[ERROR] [create-3]: process has died [pid 10254, exit code 255, cmd '/opt/ros/humble/lib/ros_gz_sim/create -topic robot_description -name my_bot -z 0.1 --ros-args'].
roar@RECARDO:~/roar_ws$
```

Solution: Exported the `IGN_GAZEBO_RESOURCE_PATH` to include the workspace models directory.

5. Invalid Classic Plugins in Ignition URDF

Issue: The terminal output stated it does not export any plugins for diff drive, camera, and IMU.



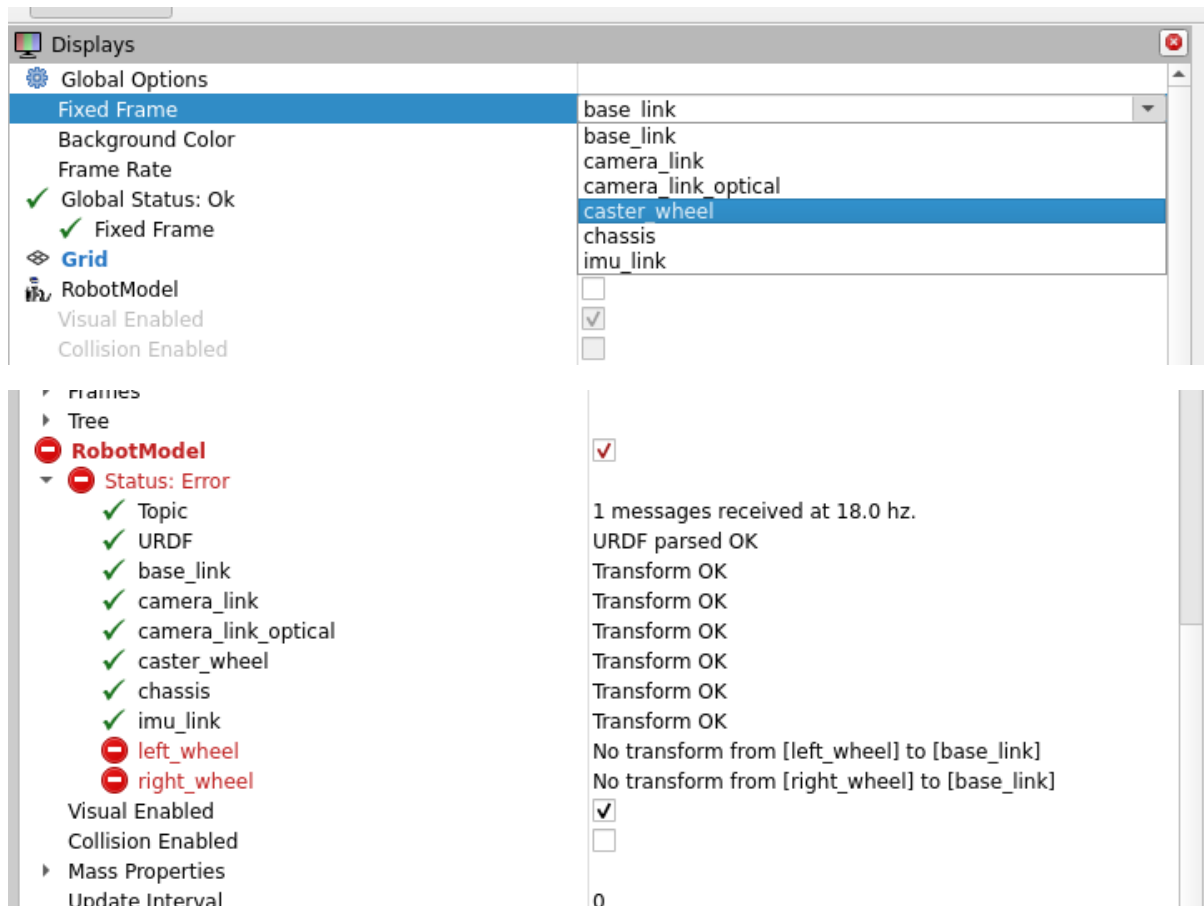
```
publisher]: got segment imu_link
[robot_state_publisher-1] [INFO] [1777576848.244705402] [robot_state_
publisher]: got segment left_wheel
[robot_state_publisher-1] [INFO] [1777576848.244708684] [robot_state_
publisher]: got segment right_wheel
[create-3] [INFO] [1777576849.185940096] [ros_gz_sim]: Waiting messag
es on topic [robot_description].
[create-3] [INFO] [1777576849.285244093] [ros_gz_sim]: Requested crea
tion of entity.
[create-3] [INFO] [1777576849.285384413] [ros_gz_sim]: OK creation of
entity.
[INFO] [create-3]: process has finished cleanly [pid 18477]
[ign gazebo-2] [Err] [SystemLoader.cc:94] Failed to load system plugi
n [libignition-gazebo-camera-system.so] : couldn't find shared librar
y.
[ign gazebo-2] [ignition::plugin::Loader::LookupPlugin] Failed to get
info for [imu_plugin]. Could not find a plugin with that name or ali
as.
[ign gazebo-2] [Err] [SystemLoader.cc:125] Failed to load system plug
in [imu_plugin] : could not instantiate from library [libignition-gaz
ebo-imu-system.so] from path [/usr/lib/x86_64-linux-gnu/ign-gazebo-6/
plugins/libignition-gazebo-imu-system.so].
```

Solution: Replaced legacy Gazebo plugins with Ignition-compatible system plugins in the URDF and initiated a `ros_gz_bridge` for `/cmd_vel` to enable teleop movement.

6. Missing Odometry and Wheel Transforms in RViz

Issue: The RViz interface was missing the `odom` fixed frame and showed transform errors for `left_wheel` and `right_wheel`.

Solution: Added the `JointStatePublisher` Ignition plugin to the URDF and executed a parameter bridge for `/joint_states`, `/tf`, and `/odom`.



7. System Plugin Loading Failure

Issue: The terminal showed Failed to load system plugin... couldn't find shared library for the IMU and Camera.

```
publisher]: got segment imu_link
[robot_state_publisher-1] [INFO] [1777576848.244705402] [robot_state_publisher]: got segment left_wheel
[robot_state_publisher-1] [INFO] [1777576848.244708684] [robot_state_publisher]: got segment right_wheel
[create-3] [INFO] [1777576849.185940096] [ros_gz_sim]: Waiting messages on topic [robot_description].
[create-3] [INFO] [1777576849.285244093] [ros_gz_sim]: Requested creation of entity.
[create-3] [INFO] [1777576849.285384413] [ros_gz_sim]: OK creation of entity.
[INFO] [create-3]: process has finished cleanly [pid 18477]
[ign gazebo-2] [Err] [SystemLoader.cc:94] Failed to load system plugin [libignition-gazebo-camera-system.so] : couldn't find shared library.
[ign gazebo-2] [ignition::plugin::Loader::LookupPlugin] Failed to get info for [imu_plugin]. Could not find a plugin with that name or alias.
[ign gazebo-2] [Err] [SystemLoader.cc:125] Failed to load system plugin [imu_plugin] : could not instantiate from library [libignition-gazebo-imu-system.so] from path [/usr/lib/x86_64-linux-gnu/ign-gazebo-6/plugins/libignition-gazebo-imu-system.so].
```

Solution: Deleted inline `<plugin>` tags from the URDF `<sensor>` blocks and defined the Sensors and Imu system plugins globally in the `.world` file.

8. XML Syntax Crash

Issue: The terminal displayed Error parsing XML... `XML_ERROR_MISMATCHED_ELEMENT` regarding the light tag.

Solution: Corrected an unclosed `<light>` tag in `aruco_world.world` that was causing the parser to abort.

9. ArUco Node Python Import Error

Issue: The terminal showed `ImportError: cannot import name 'transformations' from 'tf2.py'`.

Solution: Installed the `ros-humble-tf-transformations` package and modified the Python script to import `tf_transformations` as `transformations`.

10. NumPy 2.0 Incompatibility

Issue: The terminal showed `AttributeError: np.maximum_sctype was removed in the NumPy 2.0 release` alongside a black Gazebo window.

Solution: Downgraded NumPy to bypass the breaking change using `pip3 install "numpy<2.0.0"`.

11. OpenCV and NumPy Dependency Conflict

Issue: A pip dependency resolver error occurred because the installed OpenCV version required `NumPy ≥ 2`.

Solution: Downgraded OpenCV to align with the older NumPy requirement using `pip3 install "opencv-contrib-python<4.10"`.

12. Deprecated `np.float` Attribute

Issue: The terminal showed `AttributeError: module 'numpy' has no attribute 'float'`.

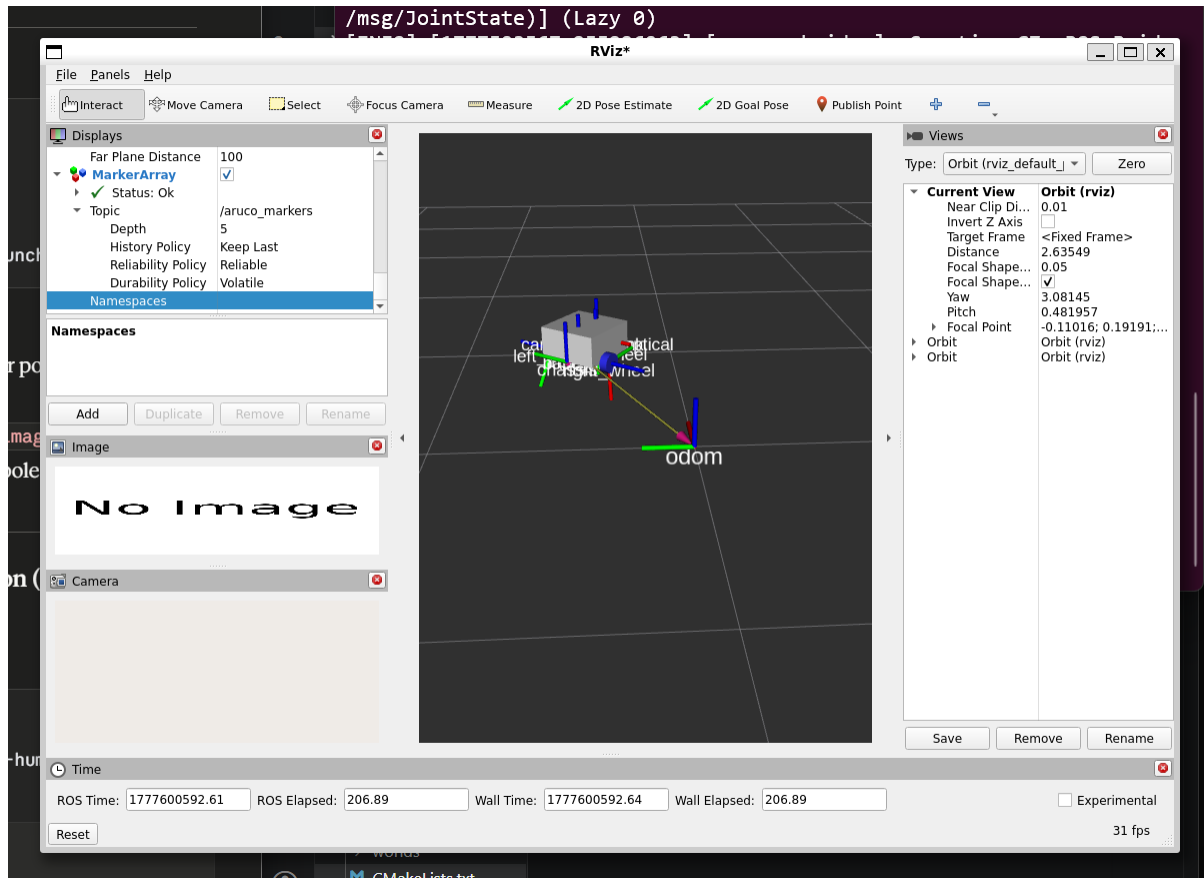
Solution: Downgraded NumPy to a legacy version that retained the deprecated float alias using `pip3 install "numpy<1.24.0"`.

13. RViz Dropping Camera Messages

Issue: The terminal output read Message Filter dropping message... the frame id of the message is empty, and RViz displayed a "No Image" placeholder.

Solution: Added `<ignition_frame_id>camera_link_optical</ignition_frame_id>` to the camera URDF, bridged the `/camera/image_raw` topic, selected the correct topic in RViz, and drove the robot to face the poles.

Supporting Figures



```
Ubuntu-22.0 x Ubuntu-22.0 x Ubuntu-22.0 x + - □ ×
x2014_x86_64.whl (17.1 MB)
17.1/17.1 MB 106.9 kB/s eta 0:00:00
Installing collected packages: numpy
  Attempting uninstall: numpy
    Found existing installation: numpy 1.26.4
    Uninstalling numpy-1.26.4:
      Successfully uninstalled numpy-1.26.4
  Successfully installed numpy-1.23.5
roar@RECARDO:~/roar_ws$ ros2 run aruco_tracking aruco_node --ros-args
-r /image_raw:=/camera/image_raw
[INFO] [1777600145.858526394] [aruco_tracking_node]: Looking for Mark
er Dictionary: 7
[INFO] [1777600145.859244565] [aruco_tracking_node]: Listening to Top
ic: /image_raw
[INFO] [1777600145.956570916] [aruco_tracking_node]: NODE STARTED SUC
CESSFULLY
[INFO] [1777601315.699699139] [aruco_tracking_node]: FOUND MARKER ID:
[54]
[INFO] [1777601315.869973343] [aruco_tracking_node]: FOUND MARKER ID:
[54]
[INFO] [1777601319.382574711] [aruco_tracking_node]: FOUND MARKER ID:
[51]
```

Figure 7: Terminal output demonstrating the resolution of dependency conflicts (NumPy downgrade) and successful ArUco node execution.

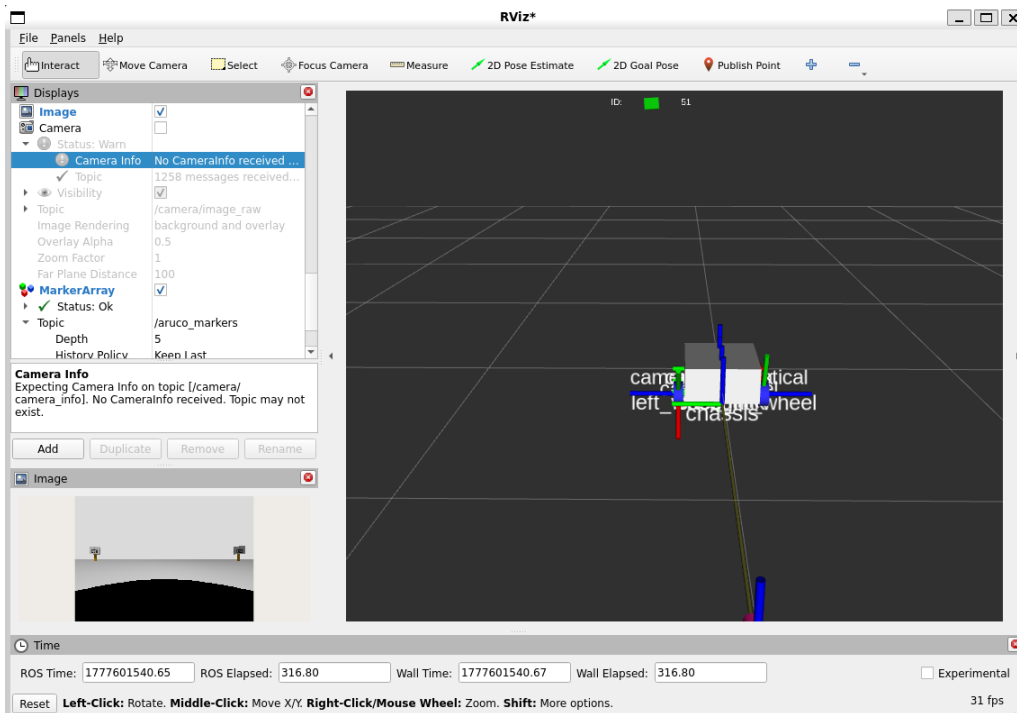


Figure 8: RViz interface validating successful marker detection, TF trees, and the resolution of dropped camera messages.

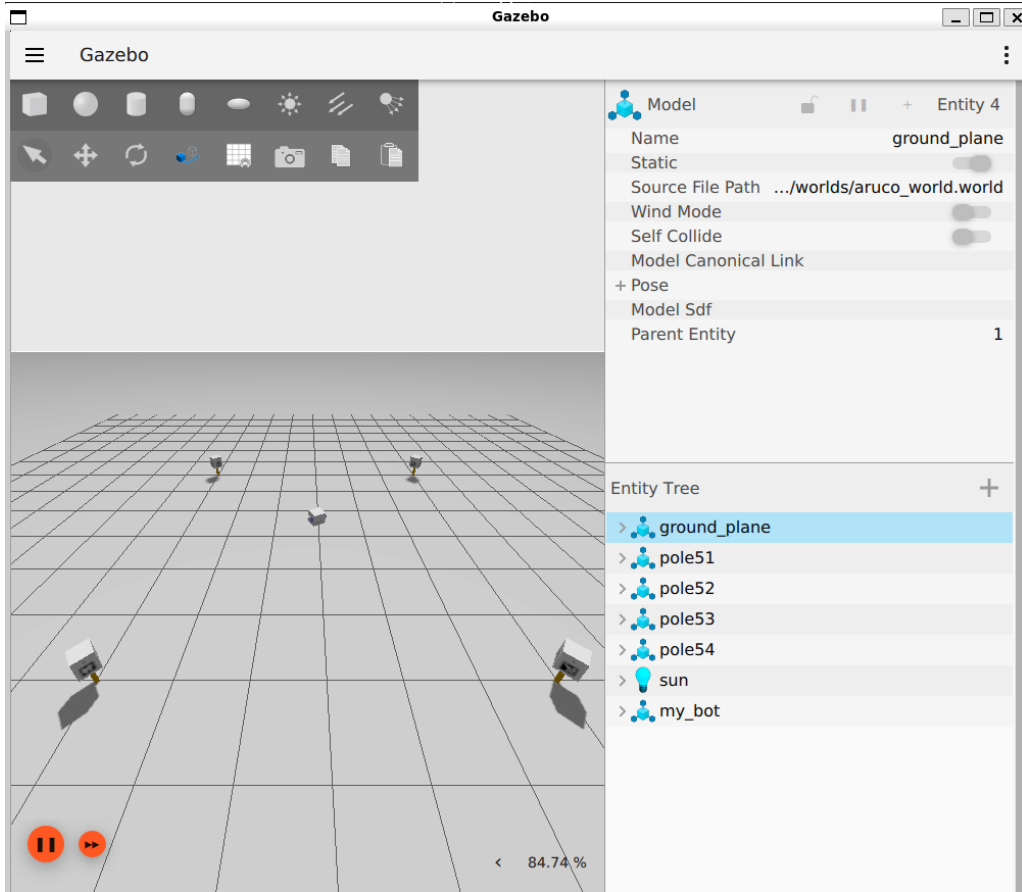
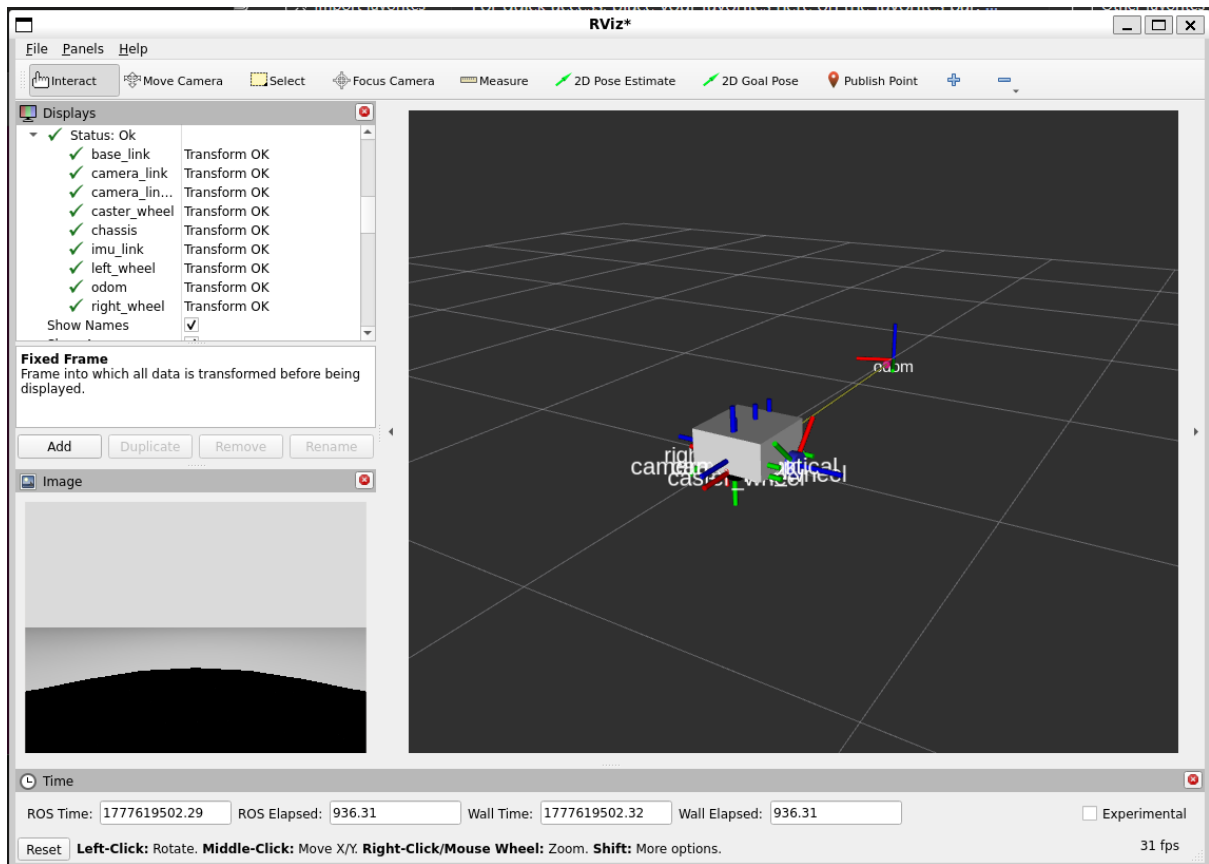


Figure 9: Gazebo Ignition interface confirming correct rendering and model loading after resolving WSL and environment path issues.

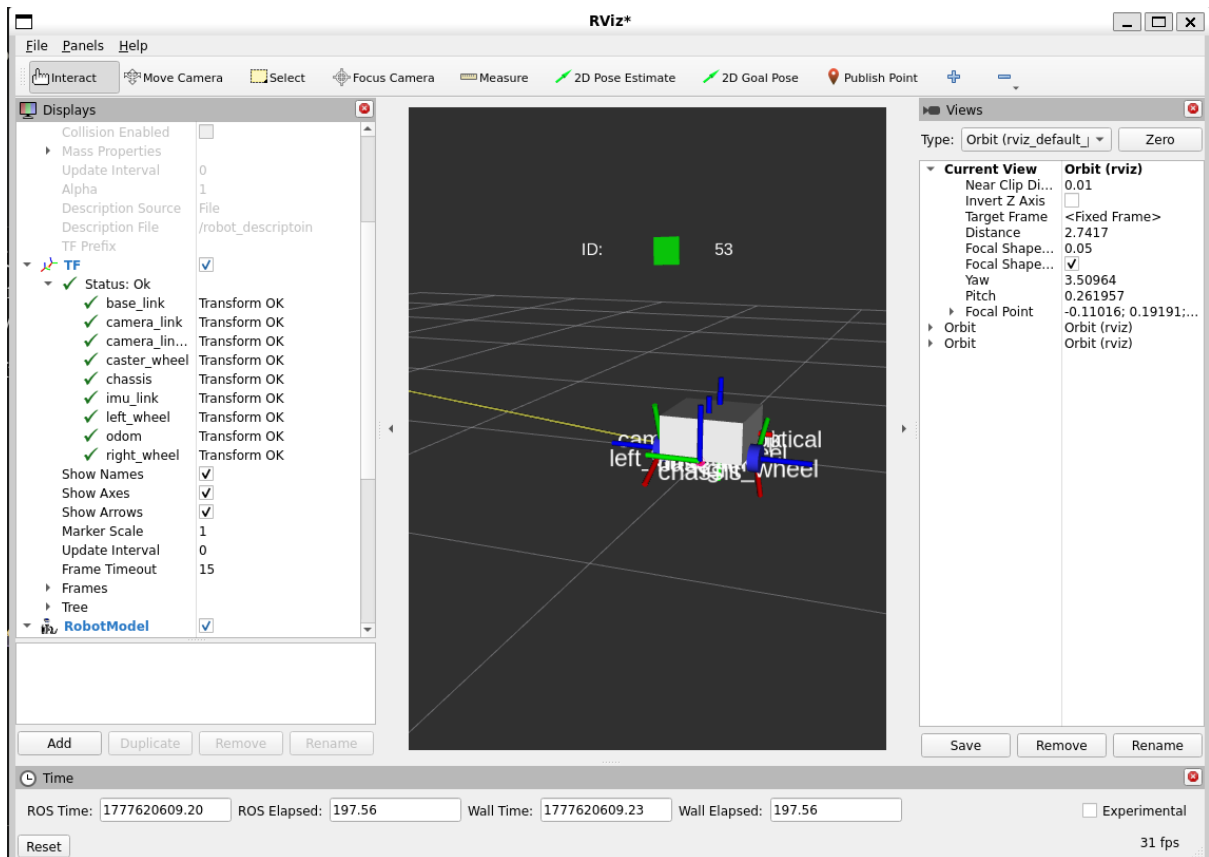
5 Perception Integration

5.1 System Implementation & Visualization

The perception module was integrated using an OpenCV-based ArUco marker detection node. The node subscribes to the simulated camera feed, processes the images to estimate the 3D poses of dictionary markers, and publishes the data as a **MarkerArray** for RViz visualization. Topic remapping was utilized to connect the Gazebo ignition camera output to the perception node's expected input.



captionRobot TF frames visible in RViz



captionDetected marker frames overlaid correctly in RViz showing 3D green cubes

5.2 Overview

This document records every error encountered during the implementation of the **Perception Integration** and **System Verification** sections of Part 2 of the ASU ROAR'26 Solo Mission. For each error the following is provided:

- The exact error message or symptom observed
- The root cause identified through debugging
- The precise terminal commands used to diagnose the issue
- The solution applied and the verification command confirming the fix

A total of **six distinct errors** were encountered and resolved across the two sections.

5.3 Error Log — Perception Integration

5.4 Error 1 — Gazebo Failed to Find `model://sun` and `model://ground_plane`

Error Details

Symptom:

Upon launching the simulation, the following errors appeared in the terminal:

```
[ign gazebo-2] [Err] Unable to find uri[model://sun]
[ign gazebo-2] [Err] Unable to find uri[model://ground_plane]
```

Resolution

Root Cause:

Ignition Gazebo does not connect to the classic Gazebo model server. The URIs `model://sun` and `model://ground_plane` failed to resolve, causing a crash.

Solution:

Replaced `model://` includes with inline SDF definitions for the sun and ground plane.

5.5 Error 2 — ArUco Pole Models Not Found

Error Details

Symptom:

```
[ign gazebo-2] [Err] Unable to find uri[model://pole51]
```

Resolution

Root Cause:

Ignition uses `IGN_GAZEBO_RESOURCE_PATH` instead of the classic `GAZEBO_MODEL_PATH`.

Solution:

Updated the launch file to export the correct environment variable.

5.6 Error 3 — ArUco Node Topic Mismatch

Error Details

Symptom:

The node produced no detections. `ros2 topic hz /image_raw` showed no publishers.

Resolution

Root Cause:

The camera plugin was publishing to `/camera/image_raw`, but the node expected `/image_raw`.

Solution:

Corrected the remapping in `sensors.xacro` to match the package requirements.

6 System Verification

This section validates the end-to-end operation of the simulated robotic system. By utilizing ROS 2 introspection tools, we verify the integrity of the transform (TF) tree, the correct establishment of node connections, and the successful flow of data from the simulated Gazebo sensors through the bridges and into the perception pipeline.

6.1 Node Graph Analysis

The following system graph demonstrates the active ROS 2 nodes and their respective topic connections. It confirms that the perception node is successfully subscribing to the bridged camera topics and publishing the processed marker array.

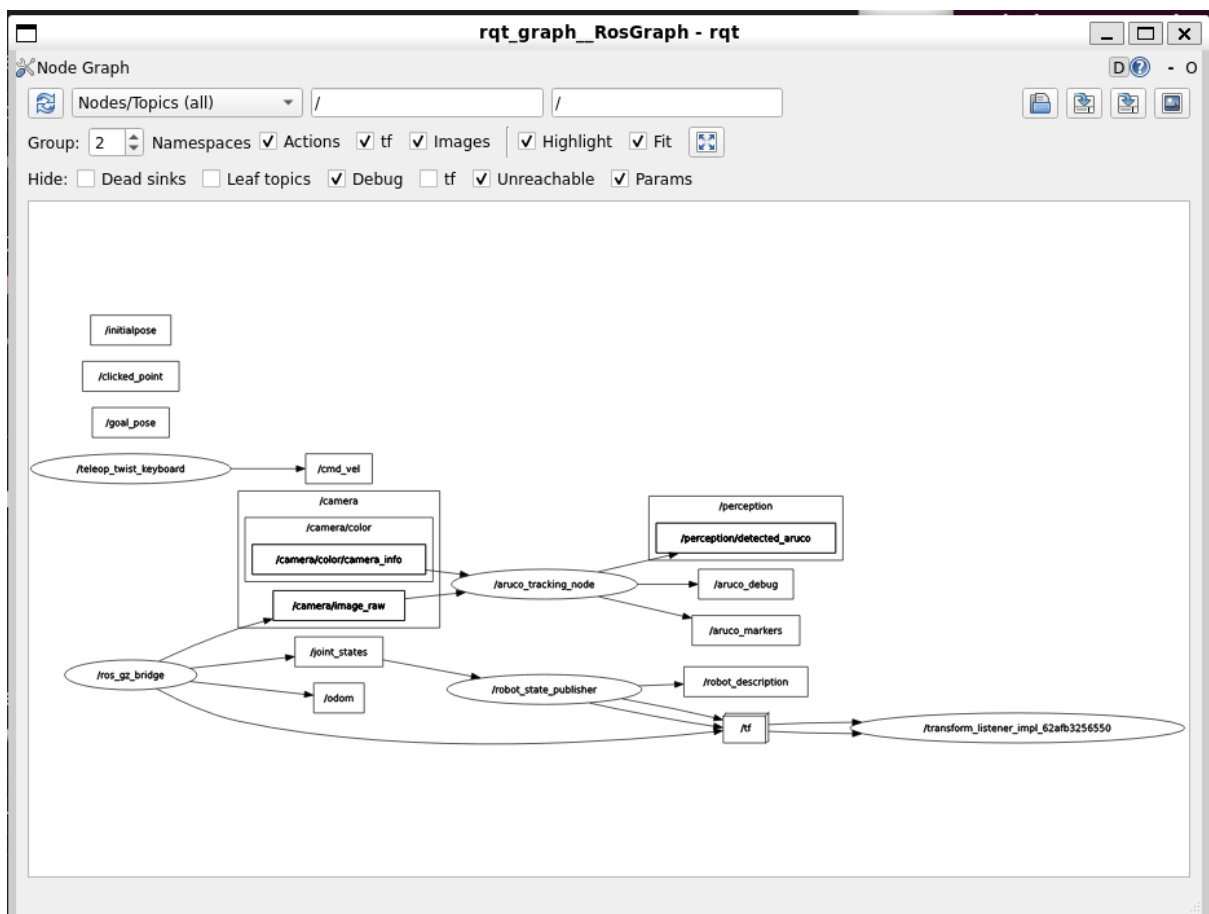


Figure 10: `rqt_graph` showing all nodes and topic connections

6.2 Transform (TF) Tree Verification

To ensure spatial consistency across the robot's links and sensors, the complete TF tree was analyzed. The tree confirms an unbroken kinematic chain from the `odom` reference frame to the `base_link`, extending outward to the individual wheel joints, the IMU, and the `camera_link_optical` frame used for visual perception.

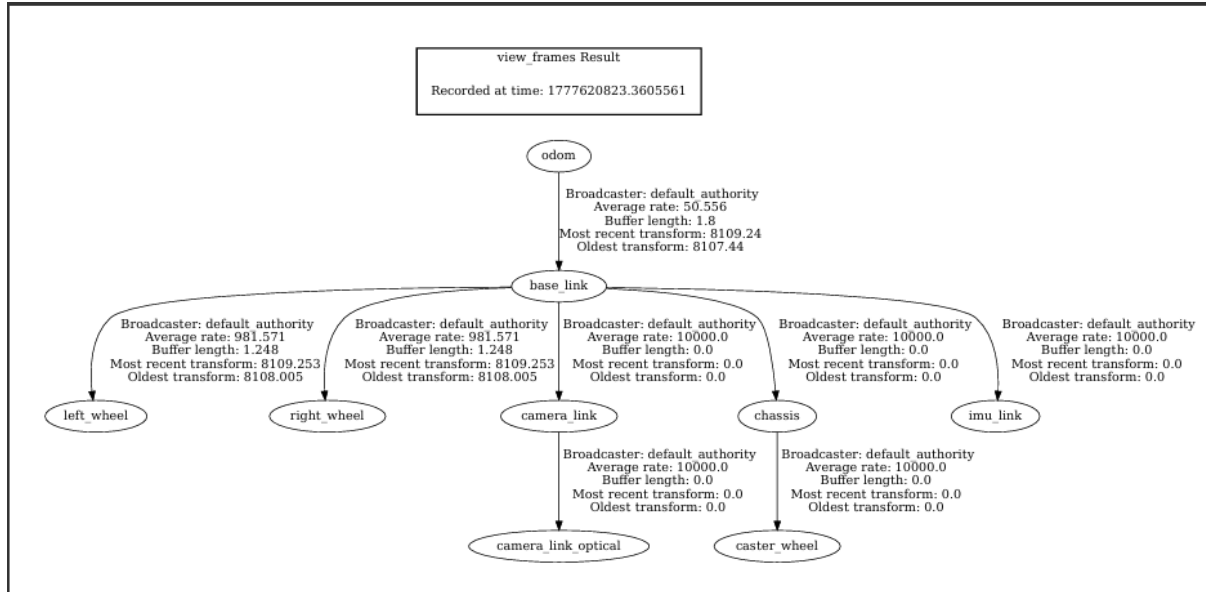


Figure 11: TF tree exported via `ros2 run tf2_tools view_frames` with no broken transforms

6.3 Sensor-to-Perception Data Flow

The final verification step confirms that raw sensor data is actively being generated, bridged, and interpreted by the system. The terminal outputs below capture the active data transmission of the sensors alongside the successful output of the ArUco detection node.

6.4 Anomaly Summary

Error Details

Anomaly 1 — Model URI errors on launch (Unable to find uri[model://sun])

Ignition Gazebo does not search the default Gazebo Classic model database. The `model://sun` and `model://ground_plane` includes fail because Ignition has no built-in model server for these URIs.

Resolution

Resolution:

Replaced both with inline SDF definitions (a `<light>` block for the sun and a `<model>` block with a plane geometry for the ground). Additionally, `IGN_GAZEBO_RESOURCE_PATH` was set in the launch file to expose the `models/` directory containing the pole models — Ignition requires this variable rather than the classic `GAZEBO_MODEL_PATH`.

Error Details

Anomaly 2 — ArUco node receiving no images (/image_raw empty)

The camera plugin was initially remapped to `/camera/image_raw` following a standard ROS 2 convention, but the provided `aruco_node` subscribes specifically to `/image_raw`. The node was receiving no data and producing no detections.

Resolution

Resolution:

Corrected the remapping in `sensors.xacro` to `image_raw:=/image_raw` and `camera_info:=/camera/color/camera_info` to match the node's expected topic names exactly, then rebuilt the workspace.

Error Details

Anomaly 3 — vision_msgs package missing

Running the `aruco` node produced `ModuleNotFoundError` or a ROS 2 dependency error related to `vision_msgs`. This package is not installed with the standard `ros-humble-desktop` meta-package.

Resolution

Resolution:

Installed the missing package via the terminal:

```
sudo apt install ros-humble-vision-msgs -y
```

Error Details

Anomaly 4 — Green marker cubes in RViz appeared at wrong position

The `MarkerArray` display showed cubes floating far from the actual pole positions. The cause was a Fixed Frame mismatch — RViz was set to `odom` while the `aruco` node published marker poses in `camera_link_optical` frame.

Resolution

Resolution:

Checked the `header.frame_id` field in `/aruco_markers` via:



Confirmed it was `camera_link_optical` and updated the RViz Fixed Frame accordingly. Marker cubes then appeared correctly positioned relative to the camera view.

7 Conclusion

The implementation of Part 2 of the ASU ROAR'26 Solo Mission successfully integrated a functional perception pipeline within a simulated environment. By transitioning the robot model to **Ignition Gazebo** and configuring the **ArUco detector node**, the system achieved real-time identification and localization of mission-critical markers.

Despite the initial technical hurdles—ranging from model URI resolution errors to ROS 2 topic mismatches—the systematic debugging approach ensured a robust final configuration. The verification phase confirmed that the robot can now perceive its surroundings, publish high-frequency sensor data, and visualize coordinate-accurate detections in RViz2, fulfilling the core objectives of the perception integration task.

8 Lessons Learned and Technical Insights

Throughout the six phases of Part 2, several key technical competencies were developed and refined:

- **Transitioning to Ignition Gazebo (Citadel/Fortress):** Learned that Ignition requires a fundamental shift from Gazebo Classic. Specifically, the necessity of using `IGN_GAZEBO_RESOURCE_PATH` for model discovery and the requirement to define world entities (like the sun and ground plane) using inline SDF or Ignition-specific URIs rather than relying on the legacy classic model server.
- **ROS 2 Perception Pipelines:** Gained a deep understanding of the ArUco detection workflow, specifically how raw images from a simulated camera plugin must be precisely mapped to the expected input topics of a subscriber node (`/image_raw` and `/camera_info`).
- **Dependency Management in ROS 2:** Recognized that meta-packages like `ros-humble-desktop` are not exhaustive. The manual installation of `vision_msgs` highlighted the importance of checking `package.xml` and identifying runtime dependency errors during node execution.
- **Coordinate Frame (TF2) Fundamentals:** Resolved visualization anomalies by understanding the relationship between the `Fixed Frame` in RViz and the `frame_id` published in the `MarkerArray` header. Correcting the frame to `camera_link_optical` was crucial for spatial accuracy.
- **Diagnostic Tool Proficiency:** Developed a standard debugging protocol using `ros2 topic hz` to verify publisher frequency, `ros2 topic echo` to inspect message headers, and `ign topic -l` to verify internal Gazebo communication before bridge integration.
- **Simulation-to-Logic Synchronization:** Learned how to bridge the gap between a physics engine (Gazebo) and ROS 2 logic, ensuring that the `ros_gz_bridge` or Ignition-native plugins correctly translate simulation events into actionable ROS 2 messages.

Resolution

Final Implementation Summary:

The mission was completed using **ROS 2 Humble** on **Ubuntu 22.04**. The final system successfully detects all four ArUco poles (51–54) at a stable 30 Hz, with 3D markers correctly projected in the global simulation frame, providing a verified foundation for future autonomous navigation tasks.